INF201 - Deliverable 3
Declaration of Generative AI Use

Scope: Tasks 1-3 (Character, Dice and Weapon classes), including the Python modules and Jupyter Notebook demonstrations.

How AI was used

I used ChatGPT as an interactive learning and debugging assistant while working on Tasks 1-3. I worked in VS Code and Jupyter Notebook, tried to write parts of the code myself, ran tests, shared screenshots and error messages, and asked for explanations and corrections. ChatGPT also supplied example code and, especially for Task 3, complete code snippets that I adapted and entered into my files. This declaration therefore does not claim that every line was written without AI assistance.

Task 1 - Character

- I asked how to define take_damage(), subtract HP, and keep HP from falling below zero using max().
- I asked how to implement heal(), learned why min() prevents HP from exceeding hp_max, and corrected attribute-name mistakes.
- I attempted the is_alive implementation and asked whether it met the assignment requirement. ChatGPT pointed out that it must be a read-only @property and helped me remove a duplicate method.
- I asked how attack() should call the target's take_damage() instead of modifying its protected HP directly.
- I requested PEP 8 formatting, type hints and docstrings, and checked the complete class through screenshots.
- I tested damage, healing, HP limits, alive/dead status and attacks in Jupyter, reporting the observed results.

Task 2 - Dice

- I asked how to construct a dice with at least two sides, why the validation checks sides < 2, and how to store self.sides.
- I wrote or attempted expressions for roll(), __repr__() and __call__(); ChatGPT explained and corrected the syntax and showed how __call__() reuses roll().
- I asked why importing Dice failed. With ChatGPT's guidance I saved the Python file, restarted the Jupyter kernel and reran the import.
- I tested six- and twenty-sided dice, direct object calls and rejection of invalid dice.

Task 3 - Weapon

- I shared the assignment requirements for items.py and asked to work faster. ChatGPT provided a full example Weapon class and suggested changes to Character.__init__() and attack().
- I asked where to document the optional weapon parameter and incorporated the explanation in the constructor docstring.
- I shared Jupyter errors about the weapon argument and an undefined Weapon name. ChatGPT helped me troubleshoot saving, restarting and running import cells in the correct order.
- I tested attacks with a weapon (random dice damage) and without a weapon (default damage of 10), and asked for concise English code comments.

Documentation and responsibility

ChatGPT also helped with the organization of Python modules versus the Jupyter Notebook, short discussion/conclusion text, and Git/GitHub instructions. I reviewed and ran the code and tests in my own environment. The submitted files and results should be assessed as my work with the AI assistance described above.

Tool: OpenAI ChatGPT. Use: explanations, code examples, debugging, documentation and workflow guidance.

INF 201 – Advanced Programming

Deliverable 3 – AI declaration

What I have used AI for

I pasted in the assignment text so that the AI could give me relevant and constructive feedback. I have solved the tasks myself and written all of the code/large parts of the code myself. I used Claude as support in the following ways:

- ***Outline:*** I asked for suggestions on how the task could be split into functions, without getting finished code.
- ***Hints when I was stuck:*** I asked specific questions about individual methods and concepts (see the list below) and used the explanations to write the code myself.
- ***Naming:*** I asked for suggestions for names of functions and variables
- ***Wording of docstrings:*** I got help wording the docstrings for the different functions in my script.
- ***Error checking and PEP8:*** I asked the AI to look over the code for errors, typos and PEP8 violations. The changes were made manually by myself.
- ***Generating a script:*** I got help from the AI to generate a script/bullet list for talking about and presenting my code in the best possible way.

I used AI to sort my questions chronologically and to word the AI declaration and the summary. Suggested changes made by the AI (Claude) were assessed and manually changed by me without copying and pasting.
The questions below are reproduced approximately and have been sorted so that they appear in the same order as in the code in the script.

Task 4: Character specialization

The assignment text and outline

1. **The assignment text**
   "Here is Task 4: (paste). I am going to make the subclasses Warrior and Mage, and my partner has already made Character, Dice and Weapon. Can you read the task and tell me what I should be aware of, without telling me how to solve it?"

The Warrior subclass

2. **Inheriting from a class**
   "How do I make a Warrior that inherits everything from Character? What does Warrior get automatically, and what do I have to write myself?"
3. **super().__init__**
   "Warrior should have all the attributes of Character plus armor. How can I reuse __init__ from Character instead of writing self.name = name and so on again? Explain with an example that is not from my assignment."

4. **Overriding take_damage**
   "Warrior should subtract armor from the damage before it is applied. How do I override a method in a subclass? Can I call Character's take_damage from inside the new method, so that I don't have to rewrite the code that keeps HP above 0?"
5. **When armor is larger than the damage**
   "What happens if the damage is 3 and the armor is 5? Does the damage become −2, and will the warrior then gain HP? Give me a hint on how to prevent this. My partner used max() and min() in take_damage and heal."
6. **__repr__ in the subclass**
   "When I write print(warrior), it says "Character: …". Why does this happen, and how can I get an output that also shows the armor? What is the difference between __str__ and __repr__?"

The Mage subclass

7. **Mana as a protected attribute**
   "Mage should have mana and mana_max. In lecture 8 we made _hp protected and created a read-only @property hp. Should I do the same with mana? What do I gain from that?"
8. **Choosing between a spell and a normal attack**
   "attack() should use a spell if Mage has at least 10 mana, and otherwise a normal attack. How do I structure the if/else for this? Don't write the method, just give me the order of the steps."
9. **Spell without a weapon**
   "The spell should deal double damage and not use the weapon, even if Mage has one. Which damage should be doubled, and how do I make sure the weapon is not used?"
10. **Falling back to the normal attack**
    "When Mage has too little mana, it should use the normal attack with the weapon. How can I use the attack from Character without copying the code? How do I make sure the mana is restored by 5, but never goes above mana_max?"

The notebook and testing

11. **Structure of the notebook**
    "The task says that the notebook should present each class and show which attributes and methods it has. How should I structure the part about Task 4? Which markdown and code cells do I need, and in what order? Don't write the cells, just give me an outline."
12. **The notebook does not see the changes**
    "The classes are in Character.py, and the tests are in the notebook. Why doesn't the notebook see the changes after I have saved the .py file? What do I need to do for it to read the file again?"
13. **Showing the attributes of an object**
    "How can I show the attributes of a Warrior or Mage in a cell without printing each attribute separately? Can I use __repr__ for this, and why do I have to write repr(warrior) instead of just print(warrior)?"

14. **What should I test for Mage?**
    "How can I show that mana goes down with spells, is restored with normal attacks, and gives a new spell when there is enough left? Can I use a for loop with range() and print HP and mana after each attack? How many attacks are needed to show a full cycle?"
15. **Testing that the spell ignores the weapon**
    "How can I test that a spell deals exactly 20 damage even if Mage has a weapon, while a normal attack uses the weapon? How do I work out how much damage one attack did?"
16. **Testing random damage**
    "The weapon uses a dice, so the damage is different each time. How can I use assert to test something that is random? I have used assert to check an exact number earlier."
17. **Showing that the subclass is a Character**
    "How can I show in the notebook that a Warrior is also a Character?

Error checking and PEP8

18. **NameError in the terminal**
    "I selected the Warrior class in Character.py and ran it. Then I got NameError: name 'Character' is not defined in a Python window with >>>. What is this window, and why can't it find Character when the class is in the same file?"
19. **NameError in the notebook**
    "I ran the Warrior cell and got NameError: name 'Warrior' is not defined, even though the class exists in Character.py. Above the error it says Cell In[1]. What does the number in brackets tell me about the order in which the cells have been run?"
20. **TypeError when creating a weapon**
    "I wrote Weapon("Staff, Dice(6))") and got TypeError: Weapon.__init__() missing 1 required positional argument: 'damage_dice'. How can Python think an argument is missing when I have written both the name and the dice? Give me a hint about where to look."
21. **Final review**
    "Here is my code for Warrior and Mage and the test cells in the notebook: (paste). Can you check that it meets the requirements in the assignment? Also look for PEP8 violations, typos, variable names that are not snake_case, comments that don't have two spaces before #, and docstrings that don't match the parameters. Don't rewrite the code, just point out which lines I should look at more closely."

Task 5: Running battles

The assignment text and outline

1. **The assignment text**
    "Here is Task 5: (paste). I am going to make a Battle class that manages a fight between two Character objects. Can you read the task and tell me what I should be aware of, without telling me how to solve it?"

The Battle class (battle.py)

2. **Which attributes does Battle need?**
   "In addition to hero and enemy, I want to keep track of which round it is and what happened in each round. Which attributes should I create in __init__, and what starting value should they have? Don't write the code, just give me suggestions."
3. **The order in one round**
   "In play_round() the hero should attack first, and the enemy should only strike back if it is still alive. How do I check that with the is_alive property my partner made in Task 1? Give me the steps in the right order."
4. **A long f-string over several lines**
   "The text from play_round() becomes longer than 120 characters. In deliverable 2 I wrote a regex pattern as several strings inside parentheses. Can I do the same with f-strings?"
5. **The while loop**
   "simulate_battle() should run rounds until one of them is no longer alive. How do I write the condition in the while loop so that it continues as long as both are alive? Why is while a better fit than for here?"
6. **A battle that never ends**
   "What happens if neither character can damage the other, for example two warriors whose damage is lower than the armor? How can I prevent the loop from running forever? Give me a hint, not the solution."
7. **Returning the winner**
   "How does simulate_battle() find out who won, and what should it return if no one won? How do I write the type hint when the method can return either a Character or None? My partner used Weapon | None."

The notebook and testing

8. **Structure of the notebook**
   "The task says that the notebook should show battles between different types of characters, with and without weapons. Which battles should I include to show everything the Battle class can do? Give me a list of test cells, but don't write them."
9. **Testing one round**
   "How can I test play_round() on its own before I run a full battle? Which numbers should I expect after two rounds between two normal characters without weapons?"
10. **Showing the whole battle**
    "simulate_battle() saves one line of text for each round in a list. How do I print all the lines after the battle, and how do I show who won and after how many rounds?"
11. **Working out the answer in advance**
    "Battles without weapons give the same result every time. How can I work out in advance who should win and how much HP the winner has left, so that I can check that the code is correct?"
12. **Who attacks first?**
    "Both hero and enemy have 100 HP and 10 damage, but the hero always wins. Why? How can I test whether it matters who attacks first?"

13. **Battles with weapons**
    “With weapons the damage becomes random. How can I show such a battle in the notebook, and what do I need to be careful about when I write the discussion, since the numbers change every time I run the cell?”
14. **Testing the safety stop**
    “How can I show in the notebook that the battle stops when no one can win? What should the output show then?”

Error checking and PEP8

15. **NameError after an import error**
    “After the import error I got NameError: name 'Battle' is not defined in the next cell. Why does this error appear when the problem was actually in the cell above?”
16. **Final review**
    “Here is my code in battle.py and the test cells in the notebook: (paste). Can you check that it meets the requirements in the assignment? Also look for PEP8 violations, typos, comments that don’t have two spaces before #, too many blank lines between methods, and docstrings that don’t match the parameters. Don’t rewrite the code, just point out which lines I should look at more closely.”